

Algorithms and Satisfiability

1. Satisfiability, Part I: Principles and Basic Algorithms

How to Think About What is True or False

Álvaro Torralba



AALBORG UNIVERSITET

Spring 2025

Agenda

- 1 Satisfiability
- 2 Basics
- 3 Applications
- 4 Normal Forms
- 5 Resolution
- 6 The Davis-Putnam (Logemann-Loveland) Procedure
- 7 Conclusion

The SAT Problem

SAT

Is a **propositional logic** formula ϕ **satisfiable**?

- **Propositional Logic**: Satisfiability can refer to many different logics. In this course, we focus on one of the simplest!
- **Satisfiable**: A formula is satisfiable if it is possible to find an interpretation (assignment) that makes the formula true.

→ Does there exist an assignment that makes the formula true?

Examples:

- $(x \vee y) \wedge (\neg x \vee \neg y)$, Yes! $x = 0, y = 1$
- $(x \vee y) \wedge (\neg x) \wedge (\neg y)$, No!

Our Agenda for This Topic

→ Our treatment of the topic “SAT Solving” consists of Chapters 7 and 8.

- **This Chapter:** Basic definitions and concepts; resolution; DPLL.
→ Sets up the framework. Resolution is the quintessential reasoning procedure underlying most successful solvers.
- **Chapter 2:** Clause learning; practical problem structure.
→ State-of-the-art algorithms for satisfiability in propositional logic, and an important observation about how they behave.
- **Mini-project:** SAT modulo theories (SMT)
→ Extension beyond propositional formulas!

Our Agenda for This Chapter

- **Basics:** What's the SAT problem about?
→ Introduces what our problem is about.
- **Applications:** What is all this useful for?
→ Brief description of some of the applications of SAT.
- **Resolution:** How does resolution work? What are its properties?
→ Formally introduces the most basic reasoning method.
- **The Davis-Putnam (Logemann-Loveland) Procedure:** How to systematically test satisfiability?
→ The quintessential SAT solving procedure, DPLL.

Syntax of Propositional Logic

→ Atoms Σ in propositional logic = Boolean variables.

Definition (Syntax). Let Σ be a set of atomic propositions. Then:

1. $\perp$ and $\top$ are Σ -formulas. (“False”, “True”)
2. Each $P \in \Sigma$ is a Σ -formula. (“Atom”)
3. If φ is a Σ -formula, then so is $\neg\varphi$. (“Negation”)

If φ and ψ are Σ -formulas, then so are:

4. $\varphi \wedge \psi$ (“Conjunction”)
5. $\varphi \vee \psi$ (“Disjunction”)
6. $\varphi \rightarrow \psi$ (“Implication”)
7. $\varphi \leftrightarrow \psi$ (“Equivalence”)

Notation: Atoms and negated atoms are called **literals**. Operator precedence: $\neg > \dots$ (we’ll be using brackets except for negation).

Semantics of Propositional Logic

Definition (Interpretation). Let Σ be a set of atomic propositions. An *interpretation* of Σ , also called a *truth assignment*, is a function $I : \Sigma \mapsto \{1, 0\}$. We set:

$$I \models \top$$

$$I \not\models \perp$$

$$I \models P \quad \text{iff} \quad P^I = 1$$

$$I \models \neg \varphi \quad \text{iff} \quad I \not\models \varphi$$

$$I \models \varphi \wedge \psi \quad \text{iff} \quad I \models \varphi \text{ and } I \models \psi$$

$$I \models \varphi \vee \psi \quad \text{iff} \quad I \models \varphi \text{ or } I \models \psi$$

$$I \models \varphi \rightarrow \psi \quad \text{iff} \quad \text{if } I \models \varphi, \text{ then } I \models \psi$$

$$I \models \varphi \leftrightarrow \psi \quad \text{iff} \quad I \models \varphi \text{ if and only if } I \models \psi$$

If $I \models \varphi$, we say that I *satisfies* φ , or that I is a *model* of φ . The set of all models of φ is denoted by $M(\varphi)$.

Semantics of Propositional Logic: Examples

Example

Formula: $\varphi = [(P \vee Q) \leftrightarrow (R \vee S)] \wedge [\neg(P \wedge Q) \wedge (R \wedge \neg S)]$

→ For I with $I(P) = 1, I(Q) = 1, I(R) = 0, I(S) = 0$, do we have $I \models \varphi$? No: $(P \vee Q)$ is true but $(R \vee S)$ is false, so the left-hand side of the conjunction is false and the overall formula is false.

Example

Formula: $\varphi = \text{InSatisfiabilityClass} \rightarrow \text{HavingAGreatTime}$

→ For I with $I(\text{InSatisfiabilityClass}) = 0, I(\text{HavingAGreatTime}) = 1$, do we have $I \models \varphi$? Yes: $\varphi = \psi_1 \rightarrow \psi_2$ is true iff either ψ_1 is false, or ψ_2 is true (i.e., $\psi_1 \rightarrow \psi_2$ has the same models as $\neg\psi_1 \vee \psi_2$).

Terminology

Satisfiability

A formula φ is:

- **satisfiable** if there exists I that satisfies φ .
- **unsatisfiable** if φ is not satisfiable.
- **falsifiable** if there exists I that doesn't satisfy φ .
- **valid** if $I \models \varphi$ holds for all I . We also call φ a **tautology**.

Equivalence

Formulas φ and ψ are **equivalent**, $\varphi \equiv \psi$, if $M(\varphi) = M(\psi)$.

Entailment

Formula φ **entails** ψ ($\varphi \models \psi$), if $M(\psi) \subseteq M(\varphi)$.

General Problem Solving using SAT

(some new problem)



model problem in logic $\mapsto$ use off-the-shelf SAT solver



(its solution)

- “Any problem that can be formulated as SAT.”
- Very successful using propositional logic and modern solvers for SAT! (Propositional satisfiability testing, **Chapter 8**.)

Applications

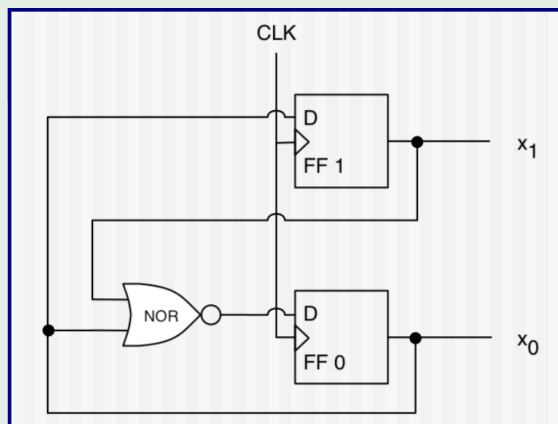
Lots of interesting problems can be formulated as SAT!

- Lots of NP problems
- Scheduling
- Hardware Verification
- Logical Deduction
- Planning (**Chapter 6**)

And we can even extend this by considering SAT modulo theories (**mini project**)

Example Application: Hardware Verification

Example



- Counter, repeatedly from $c = 0$ to $c = 2$.
- 2 bits x_1 and x_0 ; $c = 2 * x_1 + x_0$.
- (“FF” Flip-Flop, “D” Data IN, “CLK” Clock)
- **To Verify:** If $c < 3$ in current clock cycle, then $c < 3$ in next clock cycle.

Step 1: Encode into propositional logic.

- **Propositions:** x_1, x_0 ; and x'_1, x'_0 (value in next cycle).
- **Transition relation:** $x'_1 \leftrightarrow x_0$; $x'_0 \leftrightarrow \neg(x_1 \vee x_0)$.
- **Initial state:** $\neg(x_1 \wedge x_0)$. **Error property:** $x'_1 \wedge x'_0$.

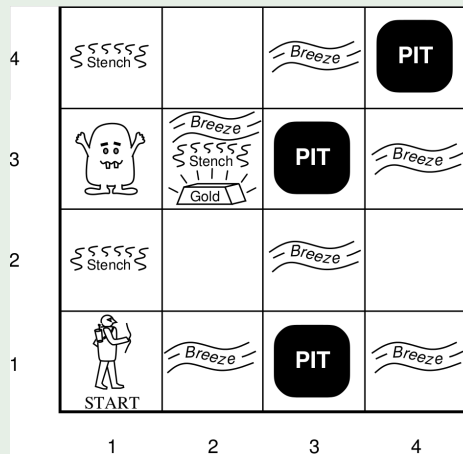
Step 2: Transform to CNF, encode as set Δ of clauses.

$\rightarrow \{\{\neg x'_1, x_0\}, \{x'_1, \neg x_0\}, \{x'_0, x_1, x_0\}, \{\neg x'_0, \neg x_1\}, \{\neg x'_0, \neg x_0\}, \{\neg x_1, \neg x_0\}, \{x'_1\}, \{x'_0\}\}$

Step 3: Call a SAT solver (up next).

Example Application: Logical Deduction (Wumpus)

Example



- The player cannot see the entire board, only if there is Stench or Breeze on the current cell.
- **To Verify:** After visiting [2,1] and [1,2], are we sure cell [2,2] is free?.

Step 1: Encode into propositional logic.

- **Propositions:** $Stench[i, j]$, $Breeze[i, j]$, $Wumpus[i, j]$, $Pit[i, j]$;
- **Knowledge Base:** $KB = \bigwedge_{i,j} Wumpus[i, j] \implies Stench[i, j + 1] \wedge Stench[1, 2] \wedge \dots$
- **Question:** $Q = \neg(Wumpus[2, 2] \wedge Pit[2, 2])$.

Step 2: Transform $KB \wedge \neg(Q)$ to CNF, encode as set Δ of clauses.

Step 3: Call a SAT solver (up next). If unsatisfiable, then we can conclude Q .

The Truth Table Method

Want: Determine whether φ is satisfiable, valid, etc.

Method: Build the **truth table**, enumerating all interpretations of Σ .

Example

Is $\varphi = ((P \vee H) \wedge \neg H) \rightarrow P$ valid?

P	H	$P \vee H$	$(P \vee H) \wedge \neg H$	$(P \vee H) \wedge \neg H \rightarrow P$
0	0	0	0	1
0	1	1	0	1
1	0	1	1	1
1	1	1	0	1

→ Yes. φ is true for all possible combinations of truth values.

→ Is this a good method for answering these questions? No! For N propositions, the truth table has 2^N rows. [Satisfiability (validity) testing is **NP**-hard (**co-NP**-hard), but that pertains to *worst-case* behavior.]

Normal Forms

The two quintessential normal forms: (there are others as well)

- A formula is in **conjunctive normal form (CNF)** if it consists of a conjunction of disjunctions of literals:

$$\bigwedge_{i=1}^n \left(\bigvee_{j=1}^{m_i} l_{i,j} \right)$$

- A formula is in **disjunctive normal form (DNF)** if it consists of a disjunction of conjunctions of literals:

$$\bigvee_{i=1}^n \left(\bigwedge_{j=1}^{m_i} l_{i,j} \right)$$

→ Given a propositional formula φ , we can in polynomial time construct a CNF/DNF formula ψ that is satisfiable if and only if φ is. (Proof omitted)

Transformation to Normal Form

CNF Transformation (DNF Transformation: Analogously)

Exploit the equivalences:

- ① $(\varphi \leftrightarrow \psi) \equiv [(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)]$ (Eliminate “ $\leftrightarrow$ ”)
- ② $(\varphi \rightarrow \psi) \equiv (\neg\varphi \vee \psi)$ (Eliminate “ $\rightarrow$ ”)
- ③ $\neg(\varphi \wedge \psi) \equiv (\neg\varphi \vee \neg\psi)$ and $\neg(\varphi \vee \psi) \equiv (\neg\varphi \wedge \neg\psi)$ (Move “ $\neg$ ” inwards)
- ④ $[(\varphi_1 \wedge \varphi_2) \vee (\psi_1 \wedge \psi_2)] \equiv [(\varphi_1 \vee \psi_1) \wedge (\varphi_2 \vee \psi_1) \wedge (\varphi_1 \vee \psi_2) \wedge (\varphi_2 \vee \psi_2)]$ (Distribute “ $\vee$ ” over “ $\wedge$ ”)

Example: $((P \vee H) \wedge \neg H) \rightarrow P$

(eliminate $\rightarrow$): $\neg((P \vee H) \wedge \neg H) \vee P$

(move $\neg$ inwards): $(\neg(P \vee H) \vee H) \vee P$ $((\neg P \wedge \neg H) \vee H) \vee P$

(distr. “ $\vee$ ” over “ $\wedge$ ”): $((\neg P \vee H) \wedge (\neg H \vee H)) \vee P$

(distr. “ $\vee$ ” over “ $\wedge$ ”): $((\neg P \vee H \vee P) \wedge (\neg H \vee H \vee P))$

→ **Note:** The formula may grow exponentially! (“Distribute” step)

→ However, **satisfiability-preserving** CNF transformation is polynomial!

Questionnaire

Question!

A CNF formula is . . .

(A): Valid iff at least one
disjunction is valid.

(B): Valid iff every disjunction is
valid.

(C): Satisfiable if at least one
disjunction is satisfiable.

(D): Satisfiable if every
disjunction is satisfiable.

→ (A): No, other parts of the global conjunction may be false under any one given interpretation.

→ (B): Yes: The CNF is a conjunction of valid formulas, so is valid itself. (Compare the CNF transformation of the example formula on slide 20).

→ (C): No since we need *all* disjuncts to be satisfied together.

→ (D): No since we need all disjuncts to be satisfied together *by the same interpretation*.

Clausal Form

→ For the remainder of this chapter, we assume that the input is a set Δ of clauses: (The same will be assumed in **Chapter 2**)

Terminology and Notation

- A **literal** l is an atom or the negation thereof (e.g., $P, \neg Q$); the negation of a literal is denoted $\bar{l}$ (e.g., $\overline{\neg Q} = Q$).
- A **clause** C is a disjunction of literals. We identify C with the set of its literals (e.g., $P \vee \neg Q$ becomes $\{P, \neg Q\}$).
- We identify a CNF formula ψ with the set Δ of its clauses (e.g., $(P \vee \neg Q) \wedge R$ becomes $\{\{P, \neg Q\}, \{R\}\}$).
- The **empty clause** is denoted $\square$.

→ An interpretation I satisfies a clause C iff there exists $l \in C$ such that $I \models l$. I satisfies Δ iff, for all $C \in \Delta$, we have $I \models C$.

Satisfiability in the Clausal Form: Rim Cases

It's normally simple ...

- E.g., I with $I(P) = 0, I(Q) = 0, I(R) = 0$ does not satisfy $\Delta = \{\{P, \neg Q\}, \{R\}\}$.

... but can be confusing in the “rim cases”:

- Does there exist I so that $I \models \square$? No, there exists no literal $l \in \square$ that we can satisfy.
- With $\Delta = \{\square\}$, does there exist I so that $I \models \Delta$? No, because we can't satisfy $\square$.
- With $\Delta = \{\}$, does there exist I so that $I \models \Delta$? Yes, because I satisfies all clauses $C \in \Delta$ (trivial as there is no clause in Δ).

Deduction

Basic Concepts in Deduction

- **Inference rule:** Rule prescribing how we can infer new formulas.
→ For example, if the KB is $\{\dots, (\varphi \rightarrow \psi), \dots, \varphi, \dots\}$ then ψ can be deduced using the inference rule $\frac{\varphi, \varphi \rightarrow \psi}{\psi}$.
- **Calculus:** Set $\mathcal{R}$ of inference rules.
- **Derivation:** φ can be **derived** from KB using $\mathcal{R}$, $\text{KB} \vdash_{\mathcal{R}} \varphi$, if starting from KB there is a sequence of applications of rules from $\mathcal{R}$, ending in φ .
- **Soundness:** $\mathcal{R}$ is **sound** if all derivable formulas do follow logically: if $\text{KB} \vdash_{\mathcal{R}} \varphi$, then $\text{KB} \models \varphi$.
- **Completeness:** $\mathcal{R}$ is **complete** if all formulas that follow logically are derivable: if $\text{KB} \models \varphi$, then $\text{KB} \vdash_{\mathcal{R}} \varphi$.

→ If $\mathcal{R}$ is sound and complete, then to check whether $\text{KB} \models \varphi$, we can check whether $\text{KB} \vdash_{\mathcal{R}} \varphi$.

The Resolution Rule

Definition (Resolution Rule). Resolution uses the following inference rule (with exclusive union $\dot{\cup}$ meaning that the two sets are disjoint):

$$\frac{C_1 \dot{\cup} \{l\}, C_2 \dot{\cup} \{\bar{l}\}}{C_1 \cup C_2}$$

If Δ contains *parent clauses* of the form $C_1 \dot{\cup} \{l\}$ and $C_2 \dot{\cup} \{\bar{l}\}$, the rule allows to add the *resolvent* clause $C_1 \cup C_2$. l and $\bar{l}$ are called the *resolution literals*.

Example: $\{P, \neg R\}$ resolves with $\{R, Q\}$ to $\{P, Q\}$.

Lemma. The resolvent follows from the parent clauses.

Proof. If $I \models C_1 \dot{\cup} \{l\}$ and $I \models C_2 \dot{\cup} \{\bar{l}\}$, then I must make at least one literal in $C_1 \cup C_2$ true.

Theorem (Soundness). If $\Delta \vdash D$, then $\Delta \models D$. (Direct from Lemma.)

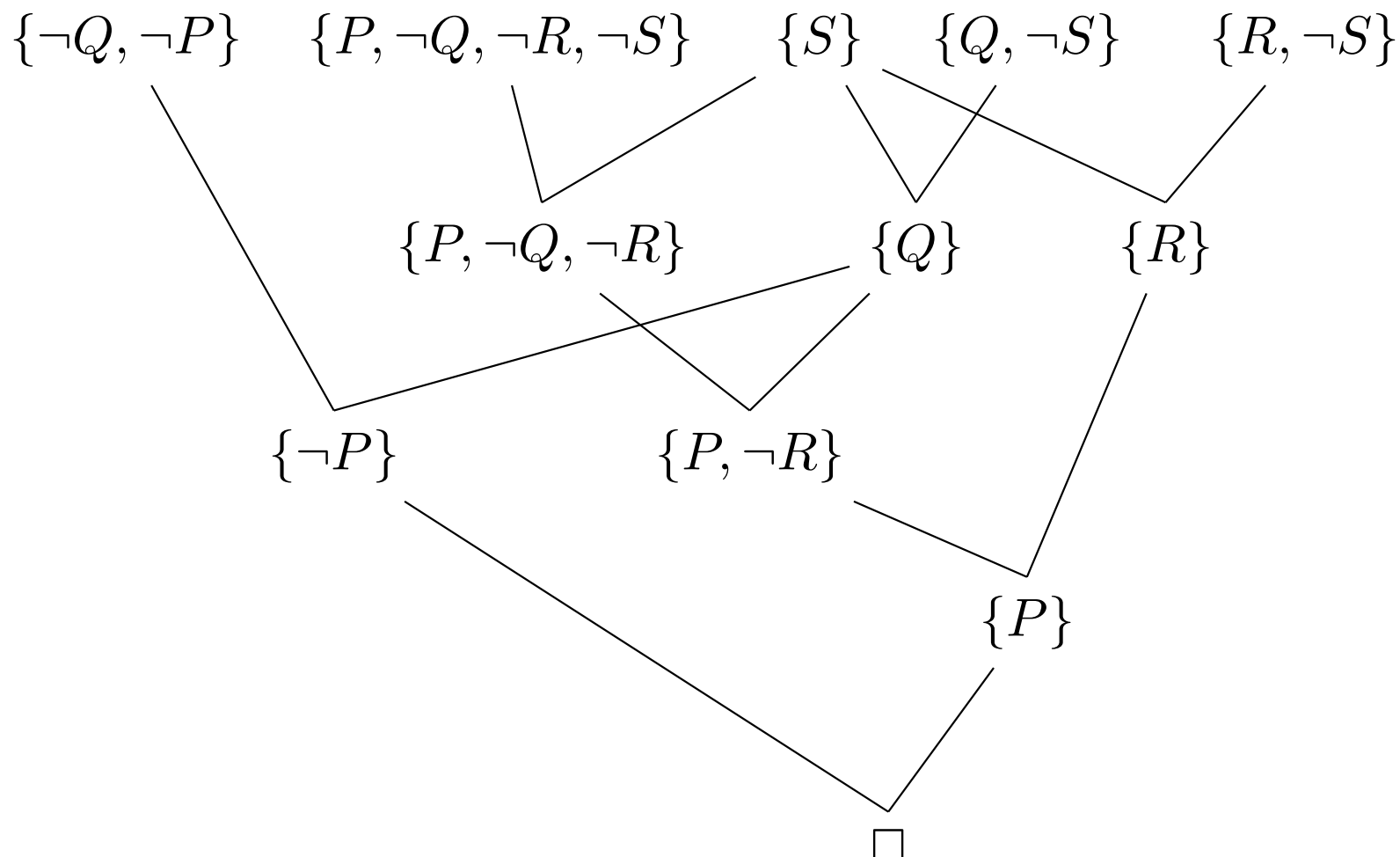
→ What about the other direction? Is the resolvent *equivalent* to its parents?

No, because to satisfy the resolvent it is enough to satisfy one of C_1, C_2 . E.g.: Setting $I(P) = 0$ and $I(Q) = 1$, we satisfy $\{P, Q\}$ but do not satisfy $\{P, \neg R\}$ when setting the resolution literal to $I(R) = 1$.

Using Resolution: A Simple Example

$\Delta = \{\{\neg Q, \neg P\}, \{P, \neg Q, \neg R, \neg S\}, \{Q, \neg S\}, \{S\}, \{R, \neg S\}\}$

Derive $\square$ by applying the resolution rule.



Using Resolution: A Frequent Mistake

Question: Given clauses $C_1 \dot{\cup} \{P, Q\}$ and $C_2 \dot{\cup} \{\neg P, \neg Q\}$, can we resolve them to $C_1 \cup C_2$?

Answer: NO!

Observation 1: Consider $\Delta = \{\{P, Q\}, \{\neg P, \neg Q\}\}$, and assume we were able to resolve as above. Then we could derive the empty clause.

However, Δ is satisfiable (e.g. $P := T, Q := F$), so this deduction would be unsound.

Observation 2: The proof of the lemma on slide 26 is not valid for the hypothetical resolution of $C_1 \dot{\cup} \{P, Q\}$ and $C_2 \dot{\cup} \{\neg P, \neg Q\}$ to $C_1 \cup C_2$.

This is due to Observation 1: An interpretation can set, e.g., $P := T, Q := F$, satisfying *both* $\{P, Q\}$ and $\{\neg P, \neg Q\}$ together, avoiding the need to satisfy either of C_1 or C_2 .

Questionnaire

Question!

What are resolvents of $\{P, \neg Q, R\}$ and $\{\neg P, Q, R\}$?

(A): $\{Q, \neg Q, P, R\}$.

(B): $\{P, \neg P, R, S\}$.

(C): $\{R\}$.

(D): $\{Q, \neg Q, R\}$.

→ (A): No. If we resolve on P then it disappears completely.

→ (B): No. By resolving on Q we get this clause except S , and although the larger clause always is sound as well of course, we are not allowed to deduce it by the rule.

→ (C): No. If we resolve on P then we get both Q and $\neg Q$ into the clause, similar if we resolve on Q .

→ We can resolve on only ONE literal at a time, cf. slide 28.

→ (D): Yes, this is what we get by resolving on P .

Unit Propagation: Intuition

Example

Consider the formula

$$(x) \wedge (x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee z)$$

- The only way to satisfy the formula is to assign 1 (true) to x .
- So, if we plug in $\top$ for every occurrence of x and simplify:
 - $(x \vee y)$ becomes $(\top \vee y)$ which is equivalent to $\top$ and can therefore be removed.
 - $(\neg x \vee z)$ becomes $(\perp \vee z)$ which is equivalent to z .

- The resulting formula

$$(z) \wedge (\neg y \vee z)$$

is satisfiable if and only if the original one is satisfiable.

- Note that unit propagation is again applicable (with z).

We Need to Talk about Cornercases

- Let's apply unit propagation to

$$(z) \wedge (\neg y \vee z).$$

We obtain the formula

$$(\top) \wedge (\neg y \vee \top)$$

simplifies to $\top$ (which is satisfiable).

- Now, let us apply unit propagation (with x) to

$$(x) \wedge (\neg x) \wedge (y \vee \neg z).$$

We obtain the formula

$$(\top) \wedge (\perp) \wedge (y \vee \neg z)$$

which simplifies to $(\perp) \wedge (y \vee \neg z)$ and then to $\perp$ (which is not satisfiable).

Simplification

We begin formalizing the DPLL algorithm by describing the recursive simplification of a formula:

```
1 def simplify( $\varphi$ ):
2     while  $\varphi$  changes:
3         replace all subformulas  $\neg\perp$  by  $\top$ 
4         replace all subformulas  $\neg\top$  by  $\perp$ 
5         replace all subformulas  $(\top\vee\psi)$  and  $(\psi\vee\top)$  in  $\varphi$  by  $\top$ 
6         replace all subformulas  $(\perp\vee\psi)$  and  $(\psi\vee\perp)$  in  $\varphi$  by  $\psi$ 
7         replace all subformulas  $(\perp\wedge\psi)$  and  $(\psi\wedge\perp)$  in  $\varphi$  by  $\perp$ 
8         replace all subformulas  $(\top\wedge\psi)$  and  $(\psi\wedge\top)$  in  $\varphi$  by  $\psi$ 
9     return  $\varphi$ 
```

```
1 #  $t \in \{0,1\}$ 
2 def fix( $\varphi$ , x, t)
3     replace every occurrence of x in  $\varphi$  by t
4     return simplify( $\varphi$ )
```

Simplification: Clause Representation

If we represent the formula as a set of clauses Δ , this is even simpler:

```
1 #  $\ell$  is a literal (e.g.  $x$  or  $\neg x$ )
2 def fix( $\Delta$ ,  $\ell$ )
3     for  $C$  in  $\Delta$ :
4         if  $\ell$  in  $C$ :
5             remove  $C$  from  $\Delta$ 
6         else if  $\neg\ell$  in  $C$ :
7             remove  $\neg\ell$  from  $C$ 
8     return  $\Delta$ 
```

Unit Propagation: Formally

Definition

We say that Δ contains a unit clause if there exist some C such that $|C| = 1$ (so $C = \{\ell\}$).

Now, we can describe unit propagation formally:

```
1 while  $\Delta$  contains a unit clause  $\{\ell\}$ : //Unit propagation
2    $\Delta = \text{fix}(\Delta, \ell)$ 
```

Some remarks:

- After fixing the value of some variable, new clauses may become unit so we need to check again
- Efficient implementations use “watch lists” to avoid checking the size of all clauses again and again

Pure Literals: Intuition

Consider the formula

$$(x \vee y \wedge \neg z) \wedge (x \vee \neg z) \wedge (y \vee \neg z) \wedge (\neg y \vee z).$$

- x only appears positively (i.e., $\neg x$ does not appear).
- So, we can replace x by 1 and obtain the (simplified) formula

$$(y \vee \neg z) \wedge (\neg y \vee z).$$

- The resulting formula is satisfiable if and only if the original one is satisfiable.
- If x only appears negatively (i.e., $\neg x$ appears but x does not), then we replace every occurrence of $\neg x$ with 1 and simplify.

Pure Literals: Formally

Definition

We say that Δ contains the pure literal ℓ if

- exists a clause $C \in \Delta$ such that $\ell \in C$
- there does not exist a clause $C \in \Delta$ such that $\neg\ell \in C$

→ We can immediately fix the value of the corresponding variable!

```
1 while  $\Delta$  contains a pure literal  $\ell$ : //Pure literal  
   simplification  
2    $\Delta = \text{fix}(\Delta, \ell)$ 
```

Splitting Rule: Intuition

Consider the formula

$$(x \vee \neg y) \wedge (x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee z) \wedge (\neg x \vee \neg z).$$

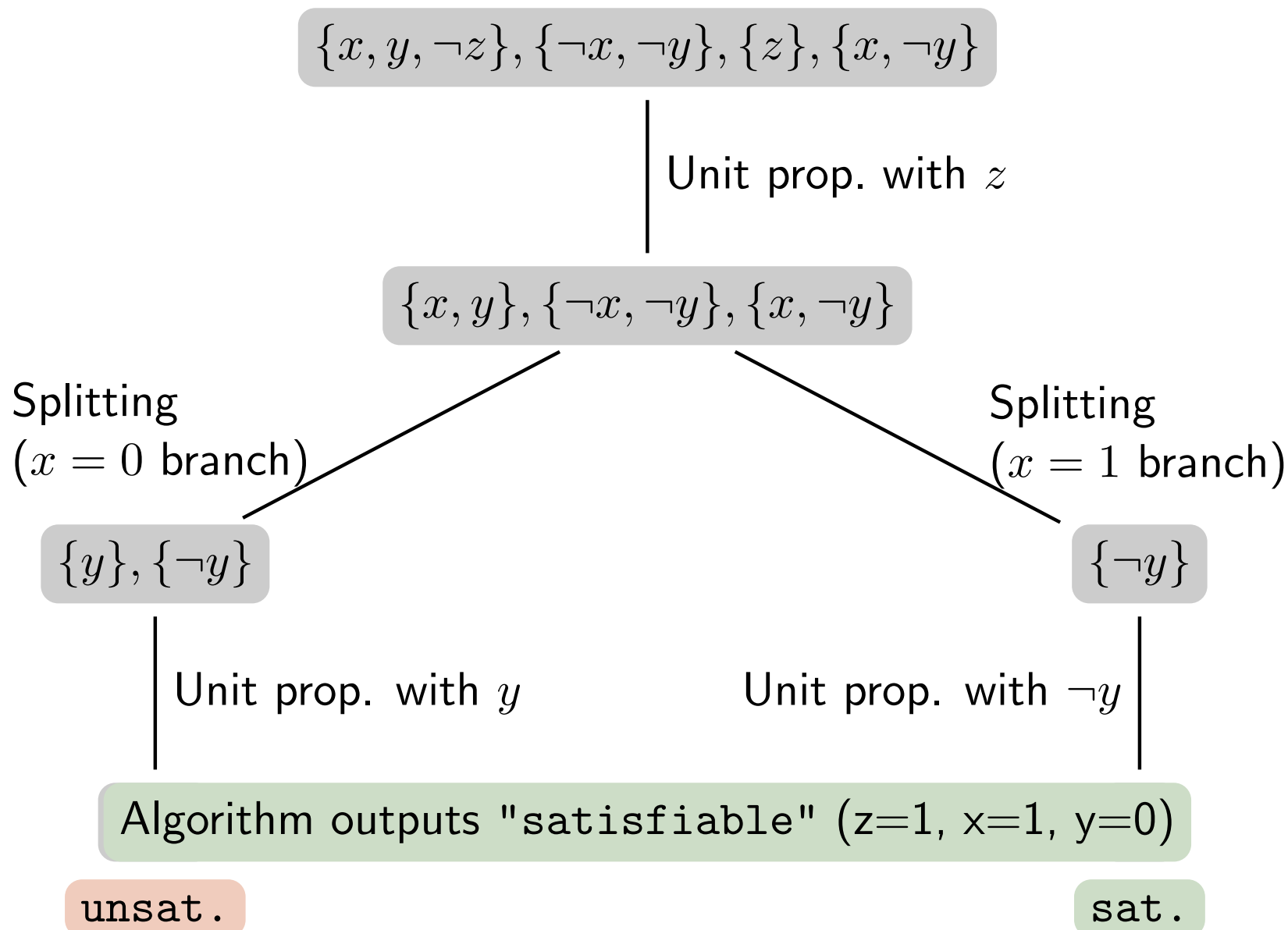
- It contains no unit clauses.
- It contains no pure literals.
- How to proceed?
- We have no other choice but to pick some variable and test both possible truth values.

Full DPLL

```
1 #  $\Delta$  is a set of clauses
2 def dpll( $\Delta$ )
3     while  $\Delta$  contains a unit clause  $\{\ell\}$ : //Unit propagation
4          $\Delta$  = fix( $\Delta$ ,  $\ell$ )
5     while  $\Delta$  contains a pure literal  $\ell$ : //Pure literal
6          $\Delta$  = fix( $\Delta$ ,  $\ell$ )
7     if  $\Delta$  = {} (contains no clause):
8         return "satisfiable"
9     if  $\square \in \Delta$  (contains the empty clause):
10        return "unsatisfiable"
11
12    // Splitting rule
13    pick some variable x in  $\Delta$ 
14    if dpll(fix( $\Delta$ ,  $\neg x$ )) == "satisfiable":
15        return satisfiable
16    else
17        return dpll(fix( $\Delta$ , x))
```

Note: A satisfying assignment can be recovered by storing all fixed literals on the path to satisfiable.

Example



Another Example

$\{\neg x, \neg y\}, \{x, \neg y, \neg z, \neg a\}, \{y, \neg a\}, \{z, \neg a\}, \{a\}$

| Unit prop. with a

$\{\neg x, \neg y\}, \{x, \neg y, \neg z\}, \{y\}, \{z\}$

| Unit prop. with y

$\{\neg x\}, \{x, \neg z\}, \{z\}$

| Unit prop. with z

$\{\neg x\}, \{x\}$

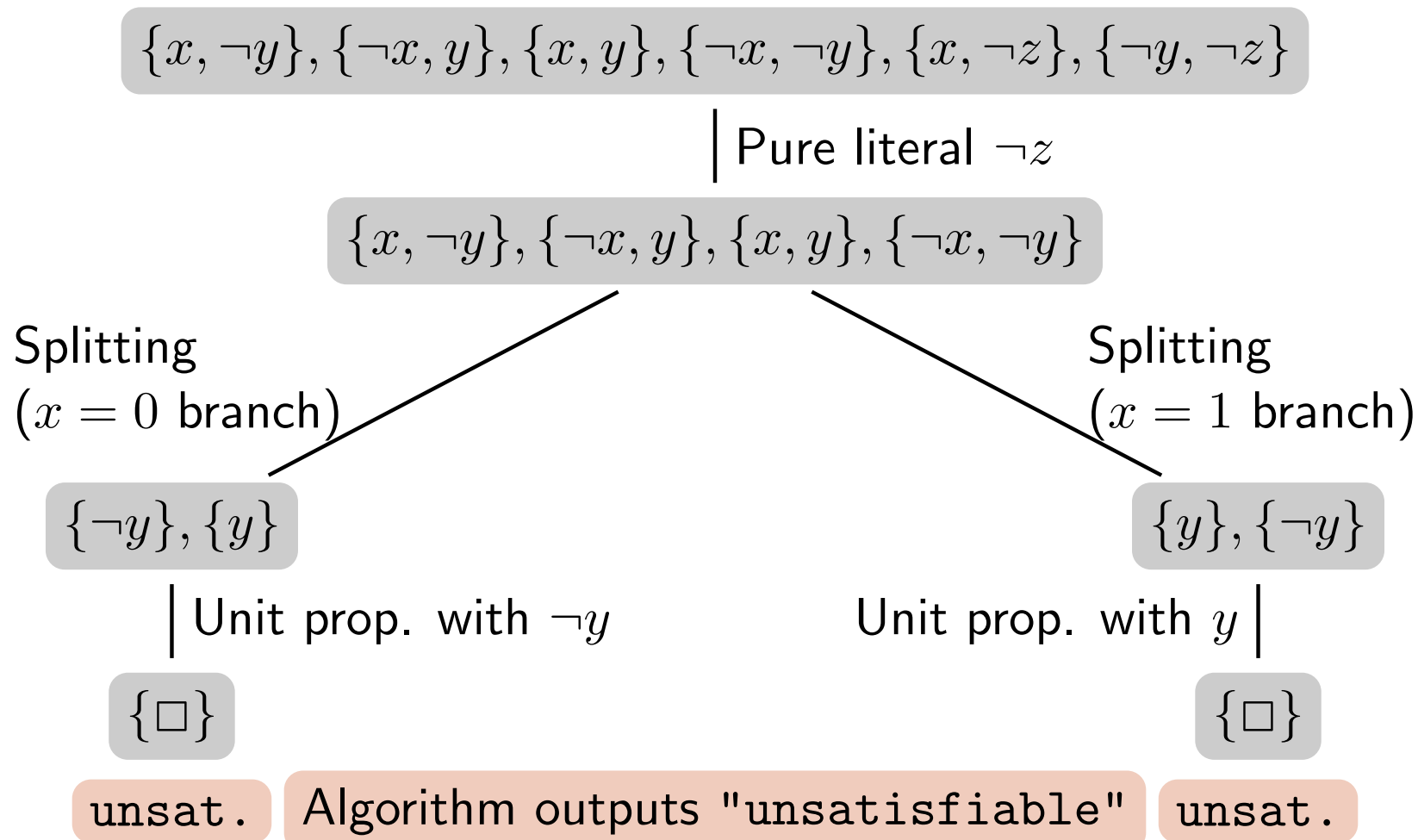
| Unit prop. with $\neg x$

$\{\square\}$

unsat.

Algorithm outputs "unsatisfiable"

And One More



Properties

Theorem

$\text{dp11}(\Delta)$ terminates after $\mathcal{O}(2^{|\Delta|})$ steps and returns "satsfiable" if and only if φ is satisfiable.

Proof.

- Runtime: at most two recursive calls, in each call at least one variable is removed, all other steps are polynomial
 $\Rightarrow$ exponential running time.
- Correctness: Based on the following two properties, which are easy to prove:
 - 1 Let ℓ be a unit clause in Δ . Then, Δ is satisfiable if and only if $\text{fix}(\Delta, \ell)$ is satisfiable.
 - 2 Let ℓ be a pure literal in Δ . Then, Δ is satisfiable if and only if $\text{fix}(\Delta, \ell)$ is satisfiable.
 - 3 Let x be a variable in Δ . Then, Δ is satisfiable if and only if $\text{fix}(\Delta, x)$ or $\text{fix}(\Delta, \neg x)$ is satisfiable.



Summary

- SAT: Is a **propositional logic** formula ϕ **satisfiable**?
 - Hard problem in general (NP-hard)
 - Many applications
- **Propositional logic** formulas are built from **atomic propositions**, with the connectives “and, or, not”.
- Every propositional formula can be brought into **conjunctive normal form (CNF)**, which can be identified with a set of **clauses**.
- **Resolution** is a deduction procedure based on trying to derive the **empty clause**. It is **refutation-complete**, and can be used to prove $KB \models \varphi$ by showing that $KB \cup \{\neg\varphi\}$ is unsatisfiable.
- **SAT solvers** decide satisfiability of CNF formulas. This can be used for deduction, and is highly successful as a general problem solving technique (e.g., in Verification).
- **DPLL** = backtracking with inference performed by **unit propagation (UP)**, which iteratively instantiates unit clauses and simplifies the formula.

Further Reading

The main material for the course are the post-handouts. If you are interested on more detailed overview of the topic, you can check these books:

- The Art of Computer Programming by Donald E. Knuth, Vol 4. Section 7.2.2.2
- Handbook of Satisfiability, Hans van Maaren, Armin Biere, Toby Walsh.